

Tugas 7: Tugas Praktikum Mandiri

Oryza Ayunda Putri - 0110224030

Teknik Informatika, STT Terpadu Nurul Fikri, Depok
E-mail: nasi.tektekmangudin@gmail.com

1. Tugas Praktikum Mandiri

1.1 Membaca Dataset

Penjelasan:

- `import pandas as pd`: Mengimpor library pandas untuk manipulasi data, diberi alias `pd` untuk memudahkan penulisan
- `pd.read_csv()`: Fungsi untuk membaca file CSV ke dalam DataFrame
- `sep=','`: Menentukan pemisah kolom adalah koma
- `df.head()`: Menampilkan 5 baris pertama data untuk preview

Python

```
import pandas as pd

#Membuat data frame untuk membaca data
df = pd.read_csv('../data/dataset_satelit.csv', sep=',')
df.head() # mengambil 5 baris data dari atas
```

	No	Longitude	Latitude	N	P	K	Ca	Mg	Fe	Mn	...	b1	Sigma_VV	Sigma_VH	plia	lia	iafe	gamma0_vv	gamma0_vh	beta0
0	1	103.036658	-0.604417	2.64	0.15	0.415	0.51	0.31	119.96	463.23	...	0.0433	0.18183	0.04461	35.74446	35.79744	35.41161	0.22331	0.05479	0.31
1	2	103.037201	-0.604689	2.75	0.17	0.568	0.76	0.58	102.63	493.81	...	0.0465	0.22079	0.04640	35.12096	35.14591	35.41510	0.27116	0.05699	0.38
2	3	103.036359	-0.603012	1.77	0.12	0.339	0.49	0.6	107.37	460.93	...	0.0417	0.18926	0.03992	35.07724	35.07730	35.41135	0.23242	0.04902	0.32
3	4	103.036950	-0.603219	2.30	0.15	0.460	0.74	0.67	96.02	338.17	...	0.0367	0.14769	0.03622	36.08078	36.08469	35.41583	0.18138	0.04448	0.25
4	5	103.036802	-0.601969	2.05	0.14	0.308	0.64	0.72	87.01	384.33	...	0.0361	0.18205	0.03797	32.68855	32.69293	35.41592	0.22359	0.04664	0.31

5 rows x 34 columns

Gambar 1 Membaca Dataset

Output:

Menampilkan tabel dengan 5 baris pertama dan 34 kolom termasuk No, Longitude, Latitude, N, P, K, Ca, Mg, Fe, Mn, dan berbagai variabel satelit lainnya.

1.2 Statistik Deskriptif

Penjelasan:

- `describe()`: Method pandas yang memberikan statistik deskriptif untuk semua kolom numerik
- Menghitung count, mean, std, min, quartiles (25%, 50%, 75%), dan max.

Python

```
df.describe()
```

	No	Longitude	Latitude	N	P	K	Ca	Fe	Mn	Cu	...	b1	Sigma_VV	Sigma_VH
count	594.000000	594.000000	594.000000	594.000000	594.000000	593.000000	594.000000	594.000000	594.000000	594.000000	...	594.000000	594.000000	594.000000
mean	297.500000	106.878644	-1.024933	2.259091	0.141380	0.582175	0.595094	74.613771	308.034697	2.391195	...	0.177291	0.234474	0.102789
std	171.617307	4.949840	0.965349	0.395499	0.019782	0.222567	0.366118	55.579655	241.731643	1.580296	...	0.155615	0.070516	0.112310
min	1.000000	102.760857	-2.333750	1.140000	0.090000	0.122000	0.050000	21.080000	3.160000	0.090000	...	0.014100	0.115170	0.021460
25%	149.250000	102.927811	-2.233338	1.982500	0.130000	0.429000	0.320000	40.705000	124.015000	1.172500	...	0.046925	0.183210	0.039535
50%	297.500000	103.581969	-0.602276	2.280000	0.140000	0.549000	0.540000	65.650000	239.445000	2.225000	...	0.072700	0.213385	0.046550
75%	445.750000	113.403797	-0.257349	2.570000	0.150000	0.710000	0.790000	87.372500	434.990000	3.357500	...	0.318900	0.262242	0.059190
max	594.000000	113.434700	0.069251	3.230000	0.220000	1.489000	2.820000	559.100000	2009.320000	8.170000	...	0.751400	0.512210	0.373000

8 rows x 33 columns

Gambar 2 Statistik Deskriptif

Output:

Tabel statistik untuk 33 kolom numerik menunjukkan:

- Count: jumlah data (594 untuk sebagian besar kolom, 593 untuk K)
- Mean: rata-rata nilai
- Std: standar deviasi
- Min/max: nilai minimum dan maksimum
- Quartiles: distribusi data

1.3 Seleksi dan Transformasi Variabel

Penjelasan:

- `df[["N", "P", "Fe"]]`: Memilih kolom N, P, dan Fe dari DataFrame
- `rename()`: Mengubah nama kolom untuk kejelasan
- `.copy()`: Membuat copy untuk menghindari modifikasi data original
- Transformasi $\times 10000$: Mengkonversi dari persen ke mg/kg (asumsi 1% = 10,000 mg/kg)
- `head()`: Menampilkan hasil transformasi

```
# pilih 2 variabel utama dari dataset
df1 = (df[["N", "P", "Fe"]]
      .rename(columns={
          "N": "nitrogen",
          "P": "fosfor"
      }).copy())

# transformasi: ubah satuan ke mg/kg (misalnya dari % ke mg/kg)
df1["nitrogen_mgkg"] = df1["nitrogen"] * 10000
df1["fosfor_mgkg"] = df1["fosfor"] * 10000

# tampilkan 5 data pertama
df1.head()
```

✓ 0.0s

	nitrogen	fosfor	Fe	nitrogen_mgkg	fosfor_mgkg
0	2.64	0.15	119.96	26400.0	1500.0
1	2.75	0.17	102.63	27500.0	1700.0
2	1.77	0.12	107.37	17700.0	1200.0
3	2.30	0.15	96.02	23000.0	1500.0
4	2.05	0.14	87.01	20500.0	1400.0

Gambar 3 Seleksi dan Transformasi Variabel

Output:

Tabel dengan 5 kolom: nitrogen, fosfor, Fe, nitrogen_mgkg, fosfor_mgkg menunjukkan data yang sudah ditransformasi.

1.4 Persiapan Data untuk Modeling

```

x = df[["N"]] # variabel independen (fitur)
y = df[["P", "Fe"]] # variabel dependen (target)

# Split data menjadi data latih dan uji (80:20)
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=7
)

```

Gambar 4 Persiapan Modeling

Penjelasan:

- `x = df[["N"]]`: Variabel independen (hanya Nitrogen)
- `y = df[["P", "Fe"]]`: Variabel dependen multipel (Fosfor dan Besi)
- `train_test_split()`: Membagi data menjadi training (80%) dan testing (20%)
- `random_state=7`: Untuk reproduktibilitas hasil
- **Catatan**: Code ini error karena `train_test_split` belum diimport

1.5 Pembuatan dan Training Model

```

from sklearn.linear_model import LinearRegression
#Buat object model instan dari class Linear Regression
model = LinearRegression()
#Lakukan proses training
model.fit(x_train, y_train)

```

0.0s

LinearRegression ⓘ ?

Parameters

Gambar 5 Training Model

Penjelasan:

- `from sklearn.linear_model import LinearRegression`: Import model regresi linear
- `LinearRegression()`: Membuat instance model regresi linear
- `model.fit(x_train, y_train)`: Melatih model dengan data training
- Model akan belajar hubungan antara `x` (N) dan `y` (P, Fe)

Output:

Menampilkan parameter model `LinearRegression` yang sudah di-fit.

1.6 Evaluasi Model

```

import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# Buat model regresi linear
model = LinearRegression()
model.fit(x_train, y_train)

# Prediksi
y_pred = model.predict(x_test)

# Evaluasi model
r2 = r2_score(y_test, y_pred, multioutput='raw_values')

print("Koefisien (pengaruh N terhadap P dan Fe):", model.coef_[0])
print("Intersep:", model.intercept_)
print("R2 (test):", r2)
print("MAE:", mean_absolute_error(y_test, y_pred))
mse = mean_squared_error(y_test, y_pred) # default squared=True
rmse = np.sqrt(mse)
print("RMSE:", rmse)

```

0.0s

Koefisien (pengaruh N terhadap P dan Fe): [0.03276132]
Intersep: [0.06718958 -37.32870823]
R2 (test): [0.44908407 0.04419325]
MAE: 11.239037781534394
RMSE: 19.550427862097845

Gambar 6 Evaluasi Model

Import library untuk evaluasi: numpy, metrics dari sklearn
 model.predict(x_test): Melakukan prediksi pada data test
 r2_score(): Menghitung koefisien determinasi R^2
 multioutput='raw_values': Mendapat R^2 untuk setiap target (P dan Fe)
 model.coef_: Koefisien regresi (slope)
 model.intercept_: Intersep model
 mean_absolute_error(): Rata-rata absolute error
 mean_squared_error() dan np.sqrt(): Menghitung RMSE

1.7 Persamaan Regresi

```

slope = model.coef_[0][0]
intercept = model.intercept_[0]
print(f"Persamaan: y = {slope:.3f} * x + {intercept:.3f}")
✓ 0.0s

```

Gambar 7 Persamaan Regresi

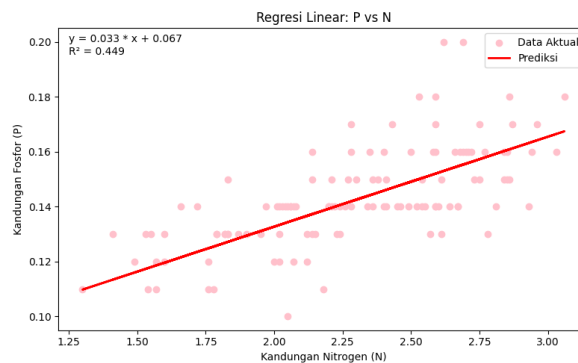
Penjelasan:

- model.coef_[0][0]: Mengambil koefisien untuk target pertama (P)
- model.intercept_[0]: Mengambil intersep untuk target pertama (P)
- f-string: Format string untuk menampilkan persamaan
- :.3f: Format 3 digit desimal

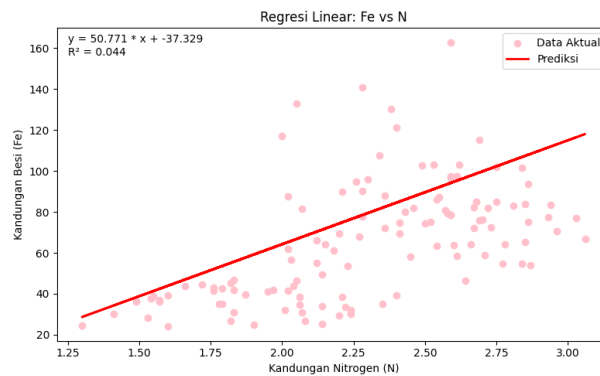
1.8 Visualisasi Hasil

Penjelasan:

- plt.figure(figsize=(8, 5)): Membuat figure dengan ukuran 8x5 inch
- plt.scatter(): Plot scatter data aktual (titik pink)
- plt.plot(): Plot garis regresi (garis merah)
- plt.xlabel(), plt.ylabel(), plt.title(): Label dan judul plot
- plt.text(): Menambahkan teks persamaan dan R^2 di plot
- transform=plt.gca().transAxes: Posisi relatif terhadap axes
- plt.legend(): Menampilkan legend
- plt.tight_layout(): Adjust layout
- plt.show(): Menampilkan plot



Gambar 8 Output Garis Regresi



Gambar 9 Output Data 2

Output:

Grafik scatter plot dengan:

- Titik pink: data aktual
- Garis merah: garis regresi
- Teks persamaan dan R^2 di sudut kiri atas

Interpretasi Hasil Akhir:

1. Koefisien [0.03276132]: Setiap kenaikan 1 unit N, P meningkat 0.033 unit
2. Intersep [0.06718958 -37.32870823]:
 - Untuk P: ketika $N=0$, $P=0.067$
 - Untuk Fe: ketika $N=0$, $Fe=-37.33$ (tidak masuk akal secara kimia)
3. R^2 [0.449 0.044]:
 - Model menjelaskan 44.9% variasi P
 - Hanya 4.4% variasi Fe yang dijelaskan
4. Error: MAE 11.24, RMSE 19.55 menunjukkan error prediksi yang cukup signifikan

Model cukup baik untuk memprediksi P berdasarkan N, tetapi sangat buruk untuk memprediksi Fe berdasarkan N.